

DRISHTITATHYA

SCANNER API INTEGRATION GUIDE

Integration Guide for the AI-Powered Packaged Commodity Compliance Scanning API

Smart India Hackathon 2026

Problem Statement: SIH26034

“दृष्टि देखे, तथ्य परखे।”

“See the Label. Verify the Facts.”

1. Introduction

1.1 Purpose of the Document

The **DrishtiTathya Scanner API Integration Guide** describes how external software systems can connect with the DrishtiTathya compliance scanning system.

The purpose of the API is to allow existing applications such as **factory management systems, warehouse systems, enterprise software, inspection applications and e-commerce platforms** to submit packaged commodity information for compliance analysis.

Instead of requiring an organization to replace its existing software, the Scanner API provides an integration layer through which DrishtiTathya's scanning and compliance capabilities can be accessed.

1.2 About DrishtiTathya

DrishtiTathya is an AI-assisted packaged commodity compliance system designed to analyze product images, extract declarations using OCR and information extraction techniques, and evaluate the extracted information against applicable Legal Metrology requirements.

The system can identify and analyze information such as:

- Product name or common/generic name
- Manufacturer, packer or importer details
- Country of origin, where applicable
- Net quantity
- Maximum Retail Price (MRP)
- Consumer care information
- Applicable declarations
- Potentially missing or inconsistent declarations

The system is designed to assist consumers, businesses and enforcement authorities in identifying potential compliance issues.

1.3 Purpose of the Scanner API

The Scanner API provides a standardized method for sending product/package information to DrishtiTathya and receiving a structured compliance result.

The basic integration process is:

Product Image → API Request → OCR → Information Extraction → Legal Rule Engine → Compliance Analysis → API Response

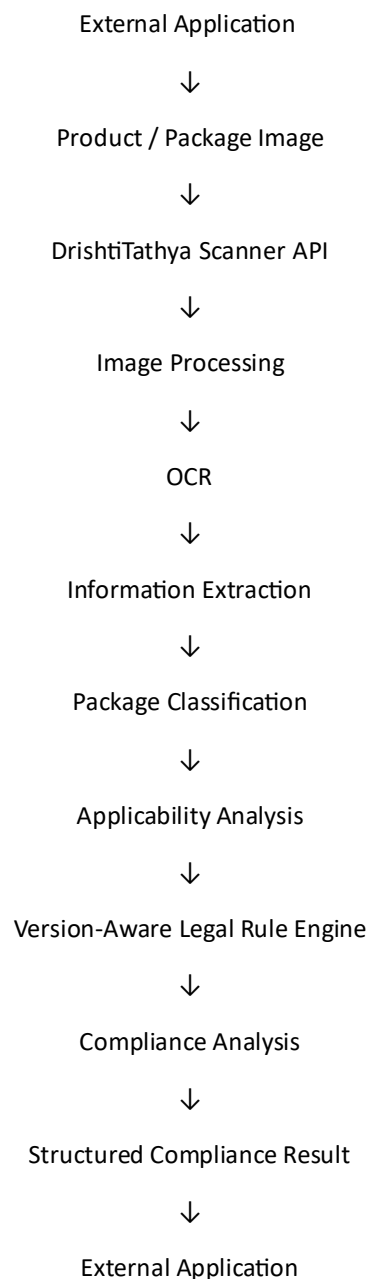
The API is intended to make DrishtiTathya's compliance capabilities reusable across different software environments.

2. Scanner API Overview

The DrishtiTathya Scanner API acts as an integration interface between external applications and the DrishtiTathya compliance analysis system.

An external application can submit a product or package image through the API. The image is then processed by the DrishtiTathya scanning pipeline.

The general processing flow is:



The API therefore separates the **external application** from the internal compliance-processing components of DrishtiTathya.

This allows organizations to integrate compliance scanning into their existing workflows without rebuilding the complete compliance system.

2.1 Basic Integration Concept

An integration can be understood as a simple request-and-response process.

Request

The external application sends information to DrishtiTathya.

For example:

- Product ID
- Product Image
- Language Preference
- Source

Processing

DrishtiTathya processes the submitted information through its scanning and compliance pipeline.

Response

The API returns structured information such as:

- Scan ID
- Processing Status
- Extracted Product Information
- Compliance Status
- Potential Violations
- Applicable Rule
- Confidence Information
- Review Requirement

3. Integration Architecture

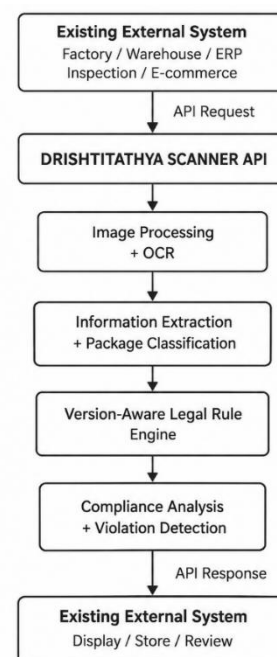
The Scanner API is designed as a service layer between an organization's existing software and the DrishtiTathya compliance engine.

3.1 Integration Benefits

The API-based architecture provides the following benefits:

- **Interoperability:** Existing software can communicate with DrishtiTathya.
- **Reusability:** The same compliance engine can support multiple applications.
- **Scalability:** Additional clients can be connected without redesigning the core system.
- **Centralized compliance logic:** Legal rules can be maintained within the DrishtiTathya rule engine.
- **Structured results:** External systems receive machine-readable compliance information.
- **Reduced duplication:** Organizations do not need to independently implement the complete compliance-analysis workflow.

4. Scanner API Endpoint



The primary scanning endpoint is proposed as:

(POST /api/v1/scan)

- **Purpose**

The endpoint accepts product/package information from an authorized external application and initiates a compliance scan.

- **HTTP Method**

- **POST**

The POST method is used because the client submits information to the DrishtiTathya system for processing.

- **Endpoint**

(/api/v1/scan)

- **Proposed Request**

```
{
  "product_id": "PC-10025",
  "image": "product_image",
  "language": "en",
  "source": "warehouse"
}
```

The exact request structure may be modified during implementation according to the final backend architecture.

4.1 Request Parameters

Parameter	Required	Description
product_id	Yes	Identifier assigned by the external system
image	Yes	Product/package image submitted for scanning
language	No	Preferred language for the response
source	No	Origin of the scan, such as factory, warehouse or mobile application

5. AUTHENTICATION & SECURITY

Security is an essential part of the **DrishtiTathya Scanner API**, because the system may process product images, extracted label information, compliance results, inspection records and evidence. The API should therefore use secure authentication and controlled access to prevent unauthorized use.

5.1 Authentication Method

DrishtiTathya APIs should use **API Key / Bearer Token authentication**.

Example:

Authorization: Bearer <API_TOKEN>

Each authorized application, department, manufacturer or integration partner should receive a unique API credential.

5.2 API Security Requirements

The following security measures are proposed:

Security Measure	Purpose
HTTPS / TLS	Encrypt data during transmission

API Key / Bearer Token	Authenticate API clients
Role-Based Access Control	Restrict access according to user role
Rate Limiting	Prevent API abuse and excessive requests
Input Validation	Prevent malformed or malicious requests
File Validation	Restrict unsupported or dangerous image files
Audit Logging	Record important API activities
Token Expiration	Reduce risk from compromised credentials
Secure Storage	Protect images, results and evidence
Access Logs	Track who accessed sensitive information

5.3 Role-Based Access

Different users should have different permissions.

Role	Major Access
Consumer	Scan products and view simplified compliance information
Manufacturer / Business	Perform self-audit and view compliance reports
Legal Metrology Officer	Conduct inspections, review evidence and confirm violations
Administrator	Manage users, rules, configurations and system operations
API Integration Client	Submit scan requests and receive compliance responses

5.4 Evidence Security

When an image is submitted for inspection, the system should maintain an evidence record containing:

- Original image reference
- Scan timestamp
- Extracted OCR text
- Detected declarations
- Rules evaluated
- Violations identified
- AI confidence score
- Review status
- User/officer information
- Result version

This creates an **audit trail** that helps determine how a compliance result was generated.

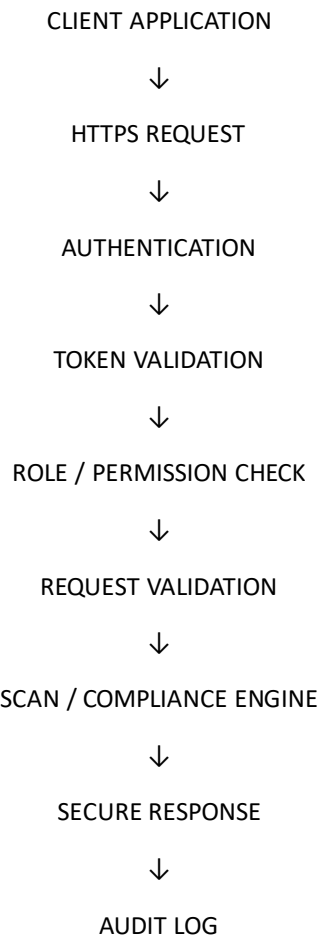
5.5 Security Principle

DrishtiTathya follows the principle:

AI assists the decision; verified evidence and authorized human review support enforcement.

The system should therefore avoid treating an AI-generated result alone as a final legal decision. Potential violations should be clearly identified and, where required, submitted for **human verification by an authorized officer**.

5.6 Recommended API Security Flow



This security architecture ensures that only authenticated and authorized systems can access the DrishtiTathya compliance services while maintaining traceability of important operations.

6. SCAN REQUEST FORMAT

The **Scan API** is the primary entry point for external applications to submit a product image or label for compliance analysis. The request should contain the image along with optional information that can improve classification and rule selection.

6.1 Basic Request

(POST /api/v1/scan)

Content-Type: multipart/form-data

Authorization: Bearer <API_TOKEN>

6.2 Request Parameters

Parameter	Type	Required	Description
image	File	Yes	Product/package image to be analysed

product_id	String	No	External system's product identifier
product_category	String	No	Known product category
country	String	No	Country/market where product is intended to be sold
language	String	No	Expected language of the label
source	String	No	Source of scan such as mobile, ERP, e-commerce or inspection
request_id	String	No	Client-generated unique request identifier

6.3 Example Request

(POST /api/v1/scan)

Authorization: Bearer <API_TOKEN>

image = product_label.jpg

product_id = PROD-10245

product_category = Food

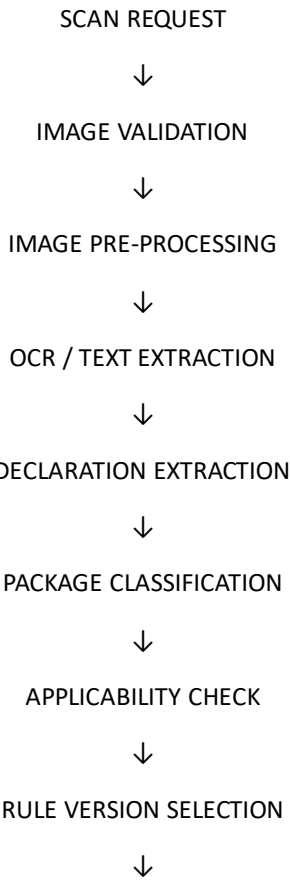
country = IN

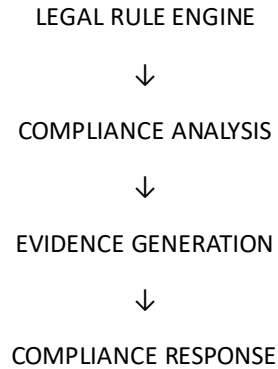
language = en

source = mobile_inspection

6.4 Processing Pipeline

After receiving the request, DrishtiTathya processes the submitted image through multiple stages:





6.5 Image Validation

Before analysis, the API should validate:

- Supported image format
- File size
- Image readability
- Image resolution
- Presence of a visible package/label
- Duplicate or corrupted uploads

If the image quality is insufficient, the system should return an appropriate warning instead of generating an unreliable compliance result.

6.6 Multiple Images

For a complete package inspection, the system may support multiple images:

Front Label + Back Label + Side Panel + Top / Bottom



COMBINED PRODUCT EVIDENCE

This is important because mandatory declarations may be distributed across different panels of a package.

6.7 Request ID and Traceability

Every request should receive a unique **Scan ID**.

Example:

SCAN-2026-00018452

The Scan ID can be used to connect:

- Submitted images
- OCR output
- Extracted declarations
- Rules evaluated
- Violations

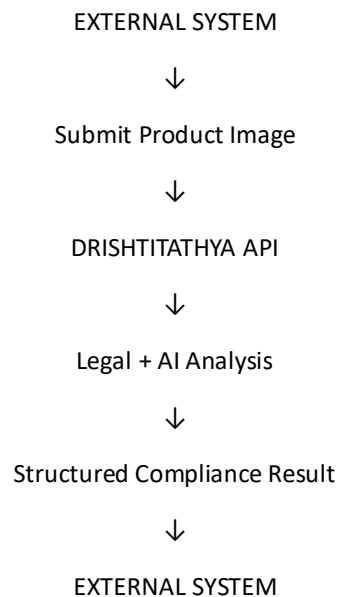
- Evidence
- Compliance response
- Human review

This provides traceability throughout the complete inspection lifecycle.

6.8 Recommended API Design Principle

The API should be designed so that an external system does not need to understand the Legal Metrology Rules internally.

Instead:



This allows DrishtiTathya to function as a **compliance intelligence layer** that can be integrated with factories, warehouses, ERP systems, inspection applications and e-commerce platforms.

7. COMPLIANCE RESPONSE FORMAT

The **Compliance Response API** returns the result of the DrishtiTathya analysis in a structured and machine-readable format. This allows external applications such as ERP systems, inspection applications, e-commerce platforms and business dashboards to directly consume the result.

7.1 Response Structure

A typical response should contain:

Field	Description
scan_id	Unique identifier of the scan
status	Overall compliance status
confidence	AI/system confidence in the analysis
product	Identified product information
declarations	Declarations extracted from the package
violations	Potential violations detected

rules_checked	Legal rules evaluated
evidence	Supporting image/text evidence
review_status	Whether human review is required
timestamp	Date and time of analysis

7.2 Example Response

```
{
  "scan_id": "SCAN-2026-00018452",
  "status": "POTENTIAL_VIOLATION",
  "confidence": 0.94,
  "product": {
    "category": "Food", "name": "Example Product"},
  "declarations": {
    "manufacturer": "ABC Foods Pvt. Ltd.",
    "net_quantity": "500 g",
    "mrp": "₹120",
    "consumer_care": "1800-XXX-XXXX",
    "country_of_origin": "India"
  },
  "violations": [
    {
      "code": "LM-R6-001",
      "field": "manufacturer_address",
      "severity": "HIGH",
      "message": "Required declaration appears incomplete",
      "confidence": 0.91
    }
  ],
  "rules_checked": [
    "Rule 4",
    "Rule 6",
    "Rule 9",
    "Rule 10"
```

```
],  
"review_status": "HUMAN_REVIEW_REQUIRED",  
"timestamp": "2026-09-14T10:30:00"}
```

7.3 Response Status

The system should use clear and standardized statuses.

- COMPLIANT : All applicable checks passed
- POTENTIAL_VIOLATION : One or more issues detected
- REVIEW_REQUIRED : AI confidence is insufficient
- UNABLE_TO_ANALYSE : Image/data quality is insufficient

7.4 Declaration Result

Each extracted declaration can also contain its own verification status.

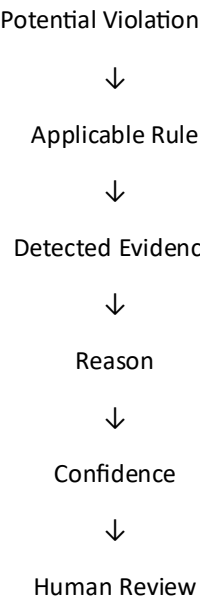
Declaration	Extracted Value	Status
Product Name	Detected	✓ Found
Manufacturer	Detected	✓ Found
Net Quantity	500 g	✓ Found
MRP	₹120	✓ Found
Consumer Care	Not detected	⚠ Missing
Country of Origin	India	✓ Found

This makes the response easier for both humans and software systems to interpret.

7.5 Explainable Compliance Result

Instead of returning only:non-compliant

DrishtiTathya should provide:



For example:

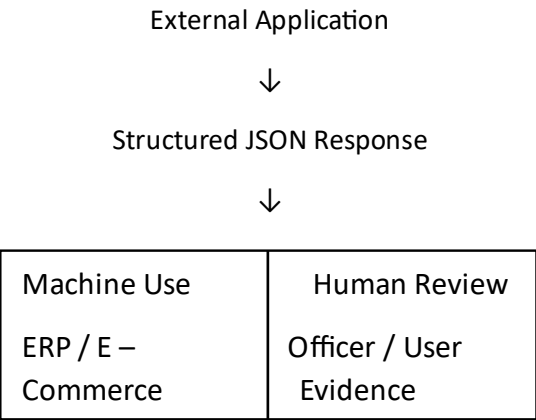
Potential Violation: Consumer-care declaration not detected.
Applicable Rule: Rule 6.

Evidence: No matching consumer-care declaration identified in the analysed package panels.
Confidence: 89%.
Action: Human verification recommended.

This approach makes the result **explainable, auditable and suitable for inspection workflows**.

7.6 API Response Principle

The response should be designed for **both machines and humans**:



Therefore, the Compliance Response API becomes the bridge between **AI-based package analysis and practical Legal Metrology compliance workflows**.

8. COMPLIANCE STATUS & VIOLATION STRUCTURE

DrishtiTathya should provide a **standardized compliance status** so that consumers, businesses, government officers and integrated software systems can easily understand the result.

The system should distinguish between an **automated finding** and a **final legal determination**. AI-generated findings should be treated as potential violations until verified by an authorized human reviewer where required.

8.1 Compliance Status

The proposed status structure is:

Status	Meaning
COMPLIANT	No potential issue detected for the applicable automated checks
POTENTIAL_VIOLATION	One or more possible compliance issues detected
REVIEW_REQUIRED	Result requires human verification because of uncertainty or legal complexity
UNABLE_TO_ANALYSE	Image/data quality or available information is insufficient
NOT_APPLICABLE	The evaluated rule or declaration does not apply to the package

8.2 Violation Structure

Every detected issue should have a structured record containing:

```
{  
  "violation_id": "VIO-000128",  
  "rule": "Rule 6",
```

```
"field": "mrp",  
"severity": "HIGH",  
"status": "POTENTIAL_VIOLATION",  
"message": "Required declaration could not be reliably verified",  
"evidence": "Front panel image",  
"confidence": 0.92,  
"review_required": true}
```

8.3 Violation Components

Component	Purpose
Violation ID	Uniquely identifies the issue
Rule Reference	Identifies the applicable legal provision
Field	Declaration or package attribute involved
Severity	Indicates priority of the issue
Message	Explains the detected problem
Evidence	Links the finding to package evidence
Confidence	Indicates AI/system confidence
Review Required	Indicates whether human verification is needed

8.4 Severity Classification

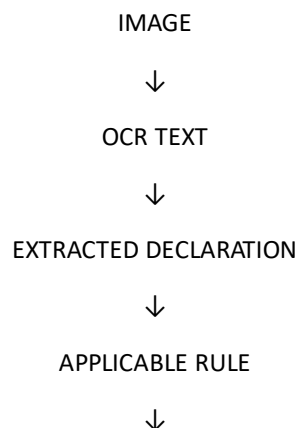
A proposed severity classification can help prioritize inspection work:

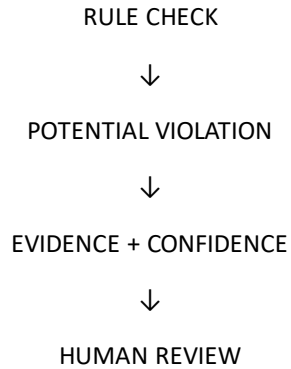
- LOW : Minor formatting/readability concern
- MEDIUM : Declaration requiring verification
- HIGH : Potentially missing or materially incorrect mandatory declaration
- CRITICAL : Serious/repeated issue requiring immediate attention according to applicable workflow

Important: Severity is a **software prioritization mechanism**, not a replacement for the legal penalty or enforcement provisions applicable to the specific case.

8.5 Evidence-Based Violation

DrishtiTathya should connect every potential violation to evidence:





For example:

Issue: Net quantity declaration could not be verified.

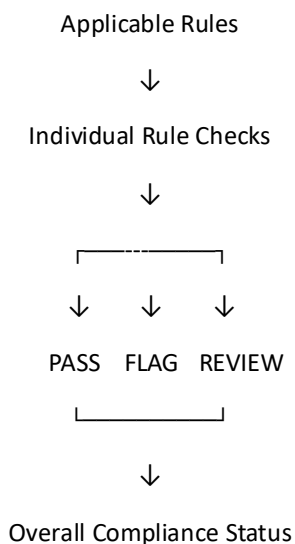
Evidence: “500 g” detected on the package.

Rule Check: Quantity declaration and applicable unit requirements evaluated.

Result: Potential issue / compliant / review required, depending on the complete package and applicable rules.

8.6 Overall Compliance Calculation

The overall result should be derived from **all applicable checks**, rather than from a single AI prediction.



This structure makes the system more transparent and allows an officer to understand **exactly why a package received a particular status**.

8.7 Human Review

When confidence is low, information is incomplete, or the issue requires legal interpretation, the system should mark:

REVIEW_REQUIRED

The reviewer can then:

- Examine the original image
- Review extracted text
- Inspect the cited rule
- Verify the potential violation

- Accept or reject the finding
- Add observations
- Record the final inspection decision

Thus, DrishtiTathya functions as an **AI-assisted compliance and inspection-support system**, rather than an autonomous legal enforcement authority.

ERROR HANDLING

The DrishtiTathya Scanner API should provide clear and standardized error handling so that external applications can understand why a request failed and take the appropriate action.

9.1 Error Categories

Error Type	Example	Suggested HTTP Status
Authentication Error	Invalid or missing API token	401
Authorization Error	Insufficient permissions	403
Invalid Request	Required parameter missing	400
Unsupported File	Invalid image format	400
File Too Large	Image exceeds allowed size	413
Poor Image Quality	Image cannot be reliably analysed	422
Processing Error	OCR/AI processing failure	500
Service Unavailable	Temporary system/infrastructure failure	503
Rate Limit Exceeded	Too many requests	429

9.2 Standard Error Response

Errors should be returned in a consistent JSON structure.

```
{
  "success": false,
  "error": {
    "code": "INVALID_IMAGE",
    "message": "The submitted image could not be processed.",
    "details": "Please upload a clear JPG, PNG or supported image.",
    "request_id": "REQ-2026-001245"
  }
}
```

9.3 Common Error Codes

The system can maintain standardized internal error codes such as:

- AUTH_INVALID_TOKEN
- ACCESS_DENIED

- INVALID_REQUEST
- IMAGE_NOT_FOUND
- INVALID_IMAGE_FORMAT
- IMAGE_TOO_LARGE
- IMAGE_LOW_QUALITY
- OCR_FAILED
- CLASSIFICATION_FAILED
- RULE_ENGINE_ERROR
- PROCESSING_TIMEOUT
- RATE_LIMIT_EXCEEDED
- INTERNAL_SERVER_ERROR
- SERVICE_UNAVAILABLE

9.4 Image Quality Handling

Image quality is particularly important for an OCR-based compliance system.

If the submitted image is blurred, too dark, cropped or does not contain sufficient package information, the system should **not generate a misleading compliance result**.

Instead:



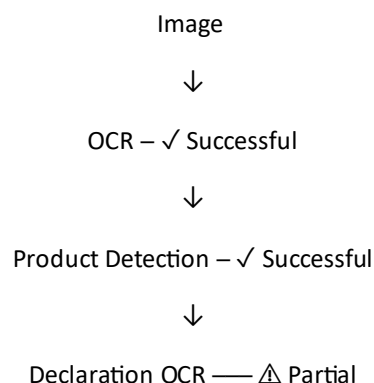
Example:

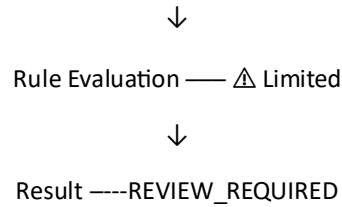
Unable to analyse: The label text is not sufficiently clear. Please capture a sharper image containing the complete declaration panel.

9.5 Partial Processing

If some information can be extracted but other information cannot, the system should preserve the available results instead of unnecessarily rejecting the entire request.

For example:

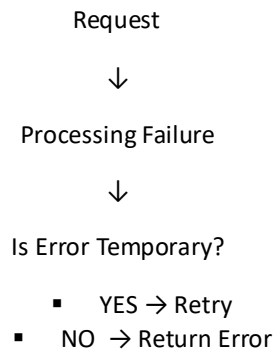




This prevents incomplete information from being incorrectly interpreted as a confirmed violation.

9.6 Retry Mechanism

Temporary failures such as network problems, service overload or processing timeouts should support controlled retries.



Repeated automatic retries should be limited to avoid unnecessary server load.

9.7 Error Logging

For every failed request, the system should record:

- Request ID
- Scan ID, if generated
- Timestamp
- Error code
- API client
- Processing stage
- Technical error information
- Retry status

Sensitive information should not be unnecessarily exposed in the response sent to external clients.

9.8 User-Friendly Error Messages

Technical errors should be translated into understandable messages for consumers and officers.

Technical:

OCR_ENGINE_TIMEOUT

User-facing:

“The label could not be analysed because text processing took too long. Please try again.”

This approach improves usability while maintaining detailed technical logs for developers and administrators.

9.9 Error Handling Principle

The core principle is:

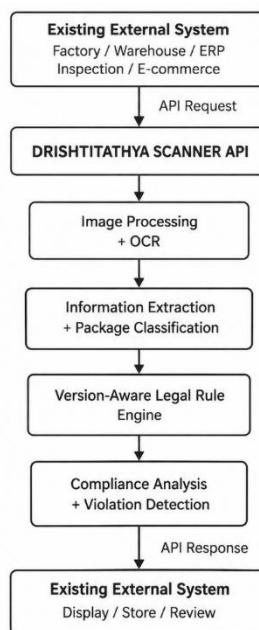
When the system is uncertain, it should report uncertainty rather than manufacture certainty.

This is especially important for DrishtiTathya because compliance analysis can support real inspection and enforcement workflows.

10. INTEGRATION WORKFLOW

The DrishtiTathya Scanner API is designed to work as a **compliance intelligence layer** that can be integrated with existing external systems such as factories, warehouses, ERP platforms, inspection applications and e-commerce platforms.

The external system does not need to implement the complete Legal Metrology rule logic. It only needs to submit the required product information to the DrishtiTathya API and consume the structured compliance response.



10.2 Step-by-Step Integration

Step 1 — Authentication

The external application authenticates with the DrishtiTathya API using its authorized credentials.

Step 2 — Submit Scan

The application sends the product image and available product metadata.

Step 3 — Image Processing

The API validates and preprocesses the image before sending it to the OCR and AI modules.

Step 4 — Information Extraction

The system identifies relevant package declarations such as:

- Manufacturer/packer/importer
- Product name
- Net quantity
- MRP
- Consumer-care information

- Country of origin
- Other applicable declarations

Step 5 — Applicability Analysis

The system determines which requirements are applicable based on available product and package information.

Step 6 — Rule Evaluation

The version-aware Legal Metrology Rule Engine evaluates the extracted information against the applicable rules.

Step 7 — Compliance Analysis

The system identifies:

- Passed checks
- Missing information
- Potential violations
- Uncertain findings
- Human-review requirements

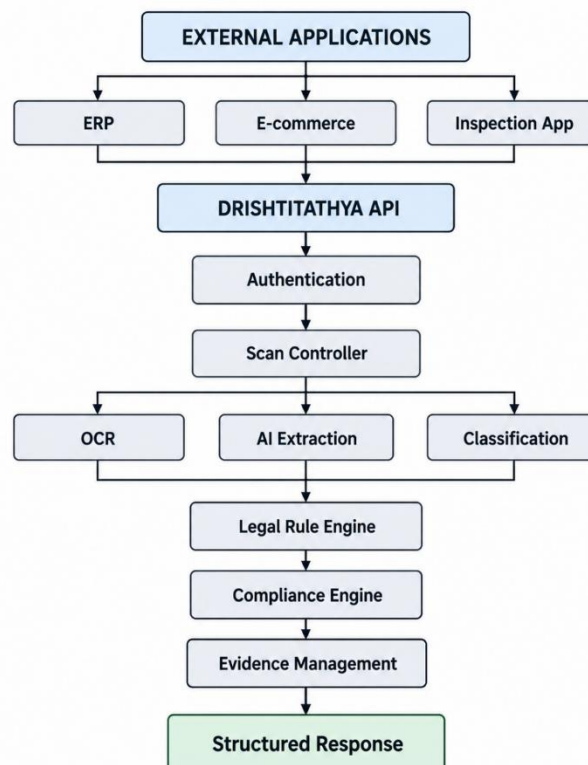
Step 8 — Response

A structured compliance response is returned to the external application.

Step 9 — Record and Audit

The scan, result and relevant evidence can be stored for future reference and audit purposes.

10.3 Integration Architecture

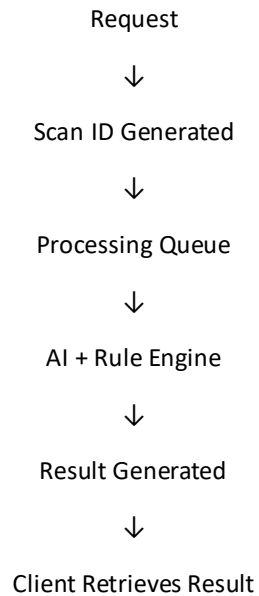


10.4 Synchronous and Asynchronous Processing

For simple scans, the system can provide a direct response:

Request → Analysis → Response

For large-scale or complex inspections, asynchronous processing can be used:



This approach allows DrishtiTathya to support both **real-time scanning** and **large-volume compliance analysis**.

10.5 Integration Principle

The API should remain **loosely coupled** with external systems.

Existing System + DrishtiTathya API = Compliance Intelligence

This means organizations can add packaged-commodity compliance capabilities without replacing their existing ERP, warehouse, inspection or e-commerce infrastructure.

10.6 Integration Security

All integrations should follow the security mechanisms defined in Section 5, including:

- HTTPS/TLS
- API authentication
- Role-based authorization
- Input validation
- Rate limiting
- Audit logging
- Secure evidence storage

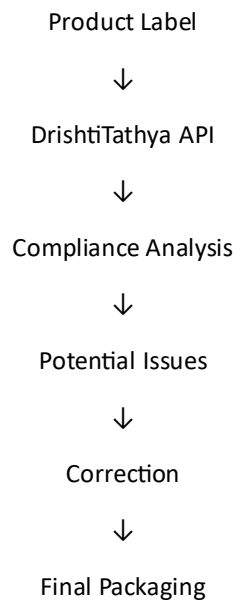
The result is a reusable architecture in which **DrishtiTathya can operate independently or become a compliance service embedded inside existing digital systems**.

11. INTEGRATION USE CASES

DrishtiTathya can be integrated into different digital environments where packaged commodities need to be checked for Legal Metrology compliance. The same Scanner API can support multiple workflows without requiring each external system to implement its own compliance engine.

11.1 Factory / Manufacturer Integration

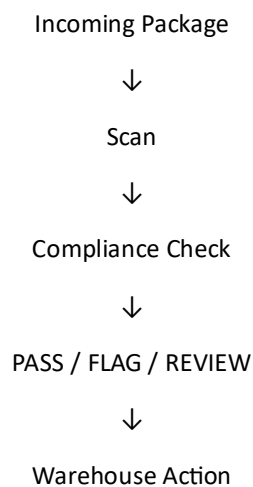
Manufacturers can integrate DrishtiTathya into their **pre-production or quality-control workflow**.



Use: Detect possible missing or incorrect declarations before products are released.

11.2 Warehouse Integration

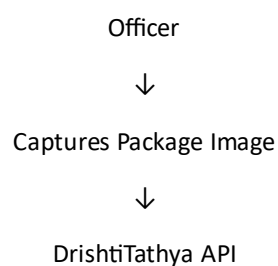
Warehouse personnel can scan packages during **receiving, storage or dispatch**.

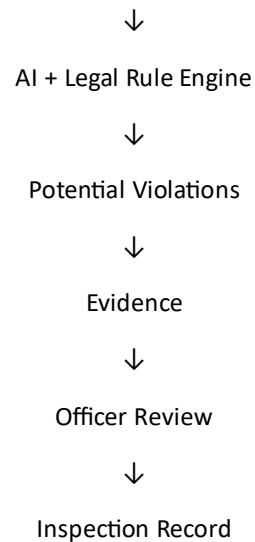


This can help identify packages requiring further verification before distribution.

11.3 Government Inspection Application

A government inspection application can use the API during field inspections.

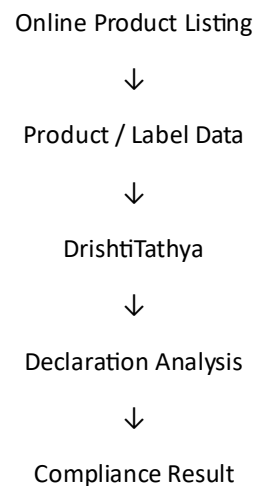




The officer can review the detected issue and make the appropriate final determination according to the applicable enforcement procedure.

11.4 E-Commerce Integration

E-commerce platforms can use DrishtiTathya to analyse product listings and package information.



Potential applications include:

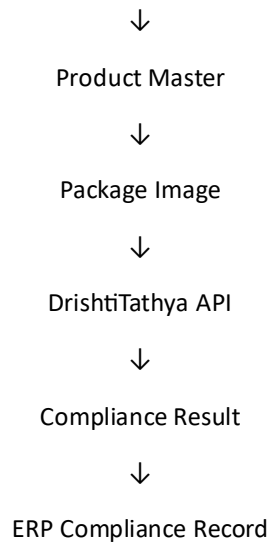
- Checking displayed mandatory declarations
- Checking country-of-origin information
- Comparing package information with online information
- Flagging listings that require verification

The architecture can also accommodate the evolving e-commerce requirements in the Packaged Commodities Rules.

11.5 ERP Integration

DrishtiTathya can be connected to an organization's ERP system.

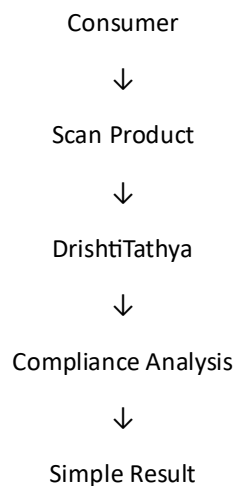
ERP



This allows compliance information to become part of the existing product lifecycle instead of remaining a separate manual activity.

11.6 Consumer Application

A consumer-facing application can provide a simplified version of the result.



Example:

✓ **No potential issue detected in automated checks**

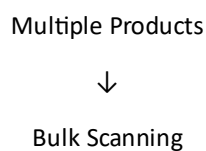
or

⚠ **Potential declaration issue detected — verify before relying on this result.**

The consumer interface should avoid presenting an AI finding as a definitive legal judgment.

11.7 Manufacturer Self-Audit

Manufacturers can perform periodic audits of multiple SKUs.

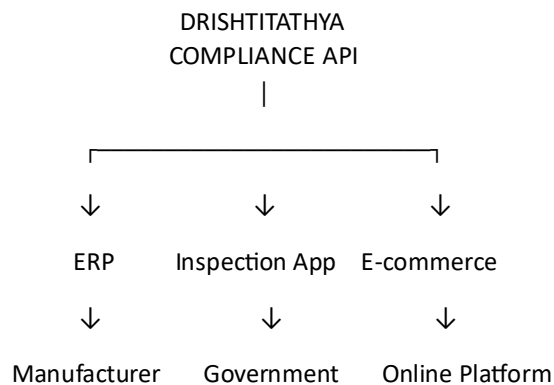




This can help businesses identify recurring packaging issues before regulatory inspection.

11.8 Cross-System Integration

The major advantage of the API architecture is that multiple systems can use the same centralized compliance intelligence.

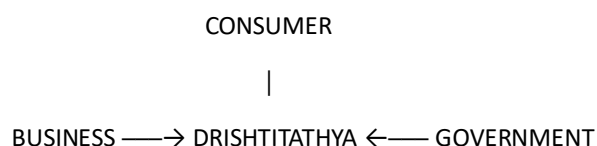


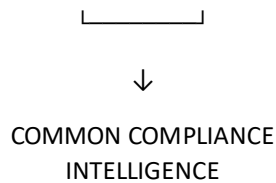
11.9 Key Integration Benefits

Benefit	Description
Reusable	One compliance engine can serve multiple applications
Scalable	Can support individual scans as well as bulk processing
Consistent	Common rule evaluation across integrated systems
Explainable	Results can include rules, evidence and reasoning
Version-Aware	Applicable rule versions can be associated with results
Auditable	Scan and decision history can be maintained
Extensible	New applications can integrate through the same API

11.10 Overall Integration Vision

DrishtiTathya is therefore not limited to a standalone mobile application. Its API architecture allows it to become a **shared compliance intelligence layer** connecting consumers, businesses and government inspection systems.



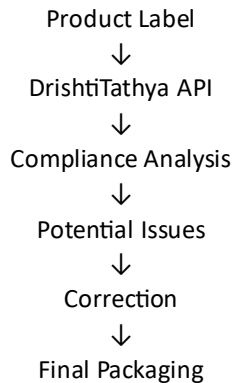


11. INTEGRATION USE CASES

DrishtiTathya can be integrated into different digital environments where packaged commodities need to be checked for Legal Metrology compliance. The same Scanner API can support multiple workflows without requiring each external system to implement its own compliance engine.

11.1 Factory / Manufacturer Integration

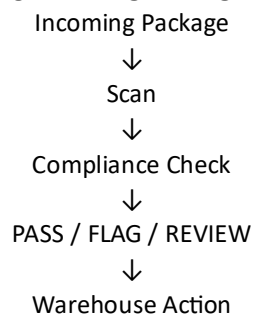
Manufacturers can integrate DrishtiTathya into their **pre-production or quality-control workflow**.



Use: Detect possible missing or incorrect declarations before products are released.

11.2 Warehouse Integration

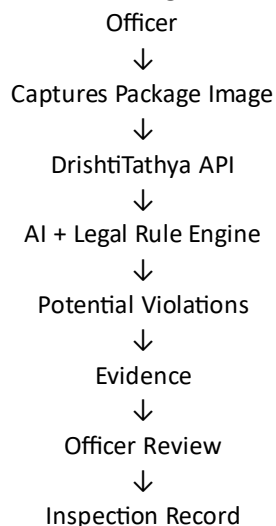
Warehouse personnel can scan packages during **receiving, storage or dispatch**.



This can help identify packages requiring further verification before distribution.

11.3 Government Inspection Application

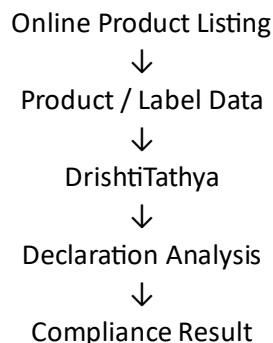
A government inspection application can use the API during field inspections.



The officer can review the detected issue and make the appropriate final determination according to the applicable enforcement procedure.

11.4 E-Commerce Integration

E-commerce platforms can use DrishtiTathya to analyse product listings and package information.



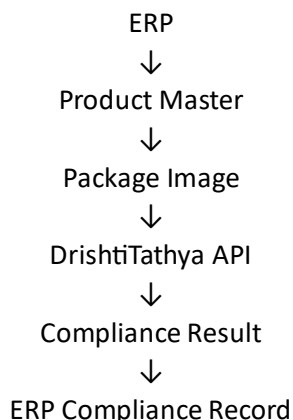
Potential applications include:

- Checking displayed mandatory declarations
- Checking country-of-origin information
- Comparing package information with online information
- Flagging listings that require verification

The architecture can also accommodate the evolving e-commerce requirements in the Packaged Commodities Rules.

11.5 ERP Integration

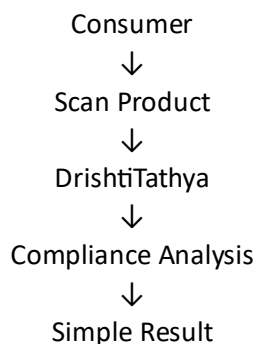
DrishtiTathya can be connected to an organization's ERP system.



This allows compliance information to become part of the existing product lifecycle instead of remaining a separate manual activity.

11.6 Consumer Application

A consumer-facing application can provide a simplified version of the result.



Example:

✓ No potential issue detected in automated checks

or

⚠ Potential declaration issue detected — verify before relying on this result.

The consumer interface should avoid presenting an AI finding as a definitive legal judgment.

11.7 Manufacturer Self-Audit

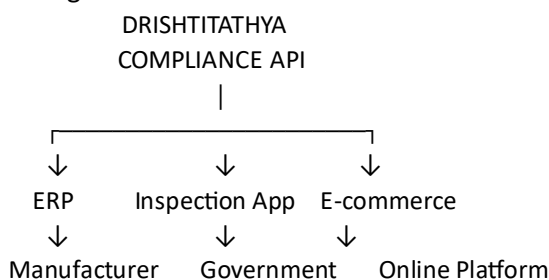
Manufacturers can perform periodic audits of multiple SKUs.



This can help businesses identify recurring packaging issues before regulatory inspection.

11.8 Cross-System Integration

The major advantage of the API architecture is that multiple systems can use the same centralized compliance intelligence.

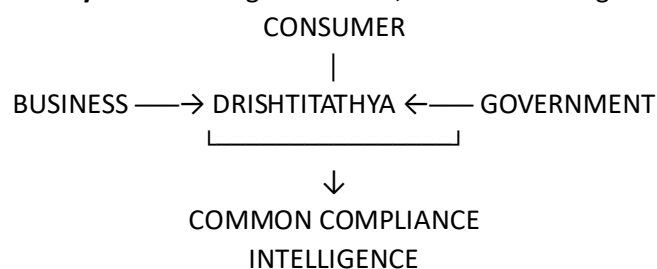


11.9 Key Integration Benefits

Benefit	Description
Reusable	One compliance engine can serve multiple applications
Scalable	Can support individual scans as well as bulk processing
Consistent	Common rule evaluation across integrated systems
Explainable	Results can include rules, evidence and reasoning
Version-Aware	Applicable rule versions can be associated with results
Auditable	Scan and decision history can be maintained
Extensible	New applications can integrate through the same API

11.10 Overall Integration Vision

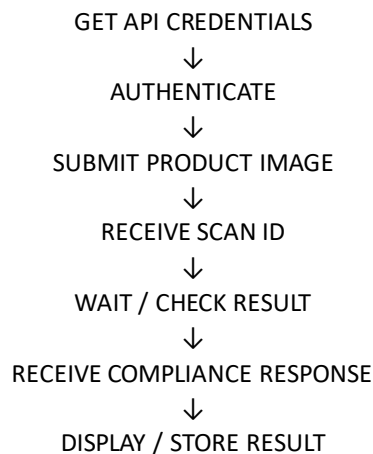
DrishtiTathya is therefore not limited to a standalone mobile application. Its API architecture allows it to become a **shared compliance intelligence layer** connecting consumers, businesses and government inspection systems.



12. DEVELOPER QUICK START

This section provides a simple procedure for developers to integrate an external application with the **DrishtiTathya Scanner API**.

The integration is designed to follow a straightforward process:



12.1 Prerequisites

Before integration, the developer should have:

- Registered API access
- API authentication credentials
- Internet connectivity
- An application capable of sending HTTPS requests
- Supported product images
- A mechanism for storing the returned Scan ID and result

12.2 Step 1 — Authenticate

Include the API token in the request header.

Authorization: Bearer <API_TOKEN>

The token should **never be hard-coded in a publicly distributed mobile application or frontend code**. It should be securely managed by the backend or authorized integration environment.

12.3 Step 2 — Submit a Scan

Send the product image to the Scan API.

(POST /api/v1/scan)

Content-Type: multipart/form-data

Authorization: Bearer <API_TOKEN>

Example conceptual request:

- image = product.jpg
- product_id = PROD-1001
- category = Food
- country = IN
- source = ERP

12.4 Step 3 — Receive the Scan ID

The API should return a unique identifier for tracking the request.

Example:

```
{
  "scan_id": "SCAN-2026-00018452",
  "status": "PROCESSING"
}
```

The external system should store this ID because it can be used to retrieve the final analysis.

12.5 Step 4 — Retrieve the Result

For asynchronous processing, the client can request the result using the Scan ID.

(GET /api/v1/scan/SCAN-2026-00018452)

Authorization: Bearer <API_TOKEN>

Example response:

```
{
  "scan_id": "SCAN-2026-00018452",
  "status": "POTENTIAL_VIOLATION",
  "confidence": 0.94,
  "review_status": "HUMAN_REVIEW_REQUIRED"
}
```

12.6 Step 5 — Display the Result

The external application can convert the structured response into its own interface.

DrishtiTathya Result

Status: Potential Violation

Issue:Manufacturer address requires
verification.

Rule:Rule 6

Confidence:91%

Action:Human Review Required

The application can also provide access to the supporting evidence and detailed compliance information.

12.7 Basic Integration Example

A backend application can conceptually implement:

```
response = submit_scan(
    image="product.jpg",
    product_id="PROD-1001"
)
scan_id = response["scan_id"]
result = get_scan_result(scan_id)
if result["status"] == "COMPLIANT":
    print("No potential issue detected")
elif result["status"] == "POTENTIAL_VIOLATION":
    print("Potential compliance issue detected")
elif result["status"] == "REVIEW_REQUIRED":
    print("Human verification required")
```

The exact SDK and programming language can be selected according to the integrating organization's technology stack.

12.8 Recommended Integration Practices

Developers should follow these practices:

1. Use HTTPS for every API request.
2. Keep API credentials secure.
3. Generate and preserve unique request IDs.
4. Handle API errors using the standard error structure.
5. Do not assume that an AI finding is a final legal determination.
6. Store Scan IDs for traceability.
7. Handle REVIEW_REQUIRED and UNABLE_TO_ANALYSE separately from violations.

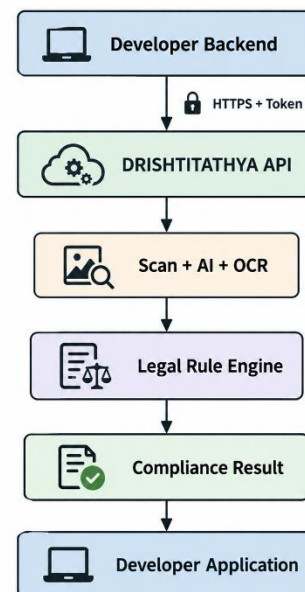
8. Implement controlled retries for temporary failures.
9. Validate uploaded images before submission.
10. Maintain audit records where the integration is used for inspection or compliance workflows.

12.9 Developer Integration Checklist

Task	Status
Obtain API credentials	☐
Configure HTTPS connection	☐
Implement authentication	☐
Implement /scan request	☐
Store Scan ID	☐
Implement result retrieval	☐
Handle compliance statuses	☐
Handle errors	☐
Display evidence/results	☐
Implement logging and monitoring	☐

12.10 Quick Architecture

The objective of the Quick Start is to make DrishtiTathya **easy to integrate without requiring developers to understand or independently implement the underlying Legal Metrology compliance rules.**



12.11 Integration Sequence

A developer can integrate DrishtiTathya by following these basic steps:

Step 1 — Get API Credentials

Obtain an authorized API token from the DrishtiTathya administration system.

Step 2 — Authenticate

Include the token in every protected API request.

Authorization: Bearer <API_TOKEN>

Step 3 — Submit Product Image

Send the product/package image to the Scan API along with optional product information.

Step 4 — Receive Scan ID

The API generates a unique Scan ID for tracking the analysis.

Step 5 — Wait / Check Result

For asynchronous processing, the application periodically checks the status using the Scan ID.

Step 6 — Receive Compliance Response

The API returns the compliance status, extracted declarations, applicable rules, potential violations, confidence and evidence references.

Step 7 — Display / Store Result

The external application displays the result to the user or stores it in its own database.

12.12 Example End-to-End Flow

Developer Application

|

1. Authenticate

|

DrishtiTathya API

|

2. Upload Image

|

Scan Controller

|

3. Generate Scan ID

|

AI + OCR Processing

|

Legal Rule Engine

|

Compliance Analysis

|

4. Result

|

DrishtiTathya API

|

Developer Application

|

Display Result

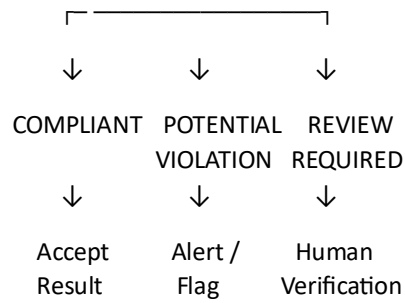
Store Result

12.13 Developer Decision Handling

The integrating application should handle each result appropriately:

COMPLIANCE RESPONSE

|



12.14 Important Developer Rule

The external application should **not interpret POTENTIAL_VIOLATION as a final legal finding**.

Instead, it should present the result as an AI-assisted compliance finding and provide access to the supporting evidence and applicable rule reference.

12.15 Quick Start Summary



This workflow allows developers to integrate DrishtiTathya with minimum changes to their existing applications while keeping **AI processing, Legal Metrology rules and compliance logic centralized within the DrishtiTathya platform**.

13. DEPLOYMENT & OPERATIONAL CONSIDERATIONS

DrishtiTathya should be designed for **reliable, scalable and secure deployment** so that it can support individual scans as well as large-scale inspection and business integrations.

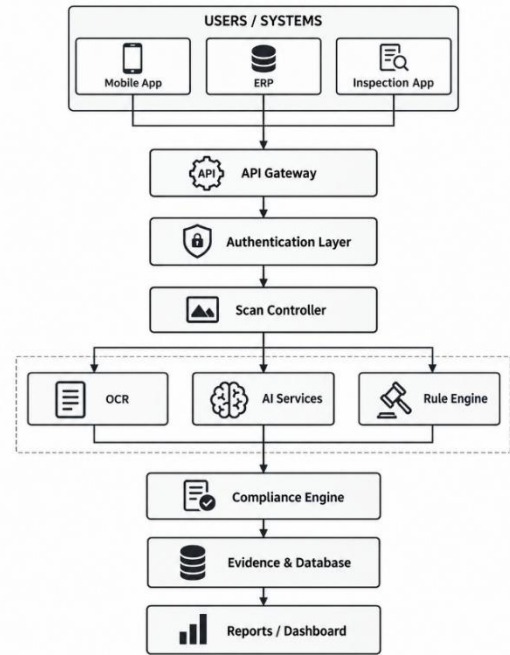
13.1 Deployment Architecture

A cloud-based deployment can provide centralized access to the API and compliance engine.

13.2 Infrastructure Requirements

The proposed system may require:

- Application/API server
- Database server
- Secure object storage for images and evidence
- OCR/AI processing infrastructure
- Rule and legal-version database
- Monitoring and logging system
- Backup and recovery infrastructure
- Secure network and API gateway



13.3 Scalability

The system should support increasing numbers of:

- Users
- Product scans
- Images
- API integrations
- Compliance rules
- Inspection records

A scalable architecture can use separate services for API handling, OCR/AI processing and rule evaluation.

Small Deployment



Single Application



Growing Demand



Separate Services



Horizontal Scaling

This allows computationally intensive AI/OCR operations to scale independently from normal API traffic.

13.4 Database and Evidence Storage

The database should maintain structured information such as:

- Users and roles
- Products
- Scan records
- Extracted declarations
- Rule versions
- Compliance results
- Violations
- Review decisions
- Audit logs

Images and other large evidence files should preferably be stored separately in secure object storage, with controlled references maintained in the database.

13.5 Monitoring and Logging

Operational monitoring should track:

Metric	Purpose
API response time	Measure system performance
Scan processing time	Monitor AI/OCR performance
API errors	Detect failures
OCR success rate	Evaluate extraction reliability
Review rate	Identify uncertain cases
Server utilization	Manage infrastructure
Storage usage	Prevent capacity issues
API request volume	Monitor system load

Alerts can be configured for significant failures or abnormal system behaviour.

13.6 Backup and Recovery

The platform should maintain regular backups of important system data.

A recovery strategy should cover:

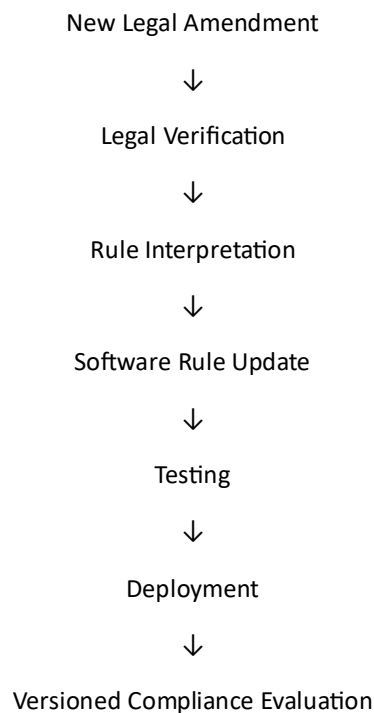
- Database failure
- Storage failure
- Application failure
- Accidental data deletion
- Infrastructure outage

Critical evidence and compliance records should have appropriate retention and recovery mechanisms.

13.7 Rule Engine Updates

A particularly important operational requirement is maintaining the **Legal Metrology rule database**.

When a new amendment becomes applicable:



Previous compliance results should remain associated with the rule version under which they were evaluated.

This prevents a later rule update from silently changing the historical interpretation of an earlier inspection.

13.8 AI Model Maintenance

OCR and AI models should be periodically evaluated using representative package images.

Maintenance may include:

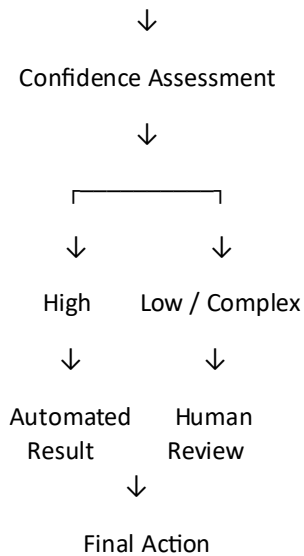
- Accuracy testing
- New packaging formats
- Different fonts and layouts
- Multilingual labels
- Low-quality image handling
- False-positive analysis
- False-negative analysis
- Model version tracking

AI model updates should be tested before production deployment.

13.9 Operational Human Review

Cases involving uncertainty should be routed to authorized reviewers.

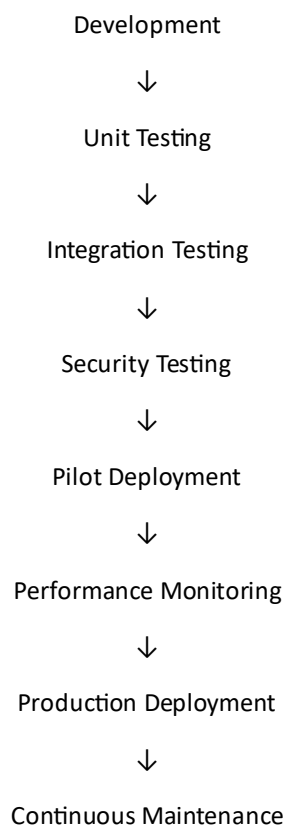
AI Analysis



This provides an important safety mechanism for legally sensitive decisions.

13.10 Production Deployment Stages

A recommended deployment process is:



13.11 Operational Governance

The system should define clear ownership for:

- Legal rule updates
- AI model updates
- API administration
- Security management

- Data retention
- Evidence access
- Incident response
- System monitoring
- Human-review workflows

13.12 Deployment Principle

The deployment architecture should ensure that **technology remains maintainable even when laws, AI models, package formats and integration requirements change.**

The most important operational principle is:

Build DrishtiTathya as a continuously maintainable compliance platform, not as a one-time scanning application.

14. CONCLUSION

DrishtiTathya is proposed as an **AI-assisted Legal Metrology compliance and inspection-support platform** for packaged commodities. The system combines product image scanning, OCR, information extraction, package classification and a version-aware legal rule engine to identify potential compliance issues.

The proposed API architecture allows DrishtiTathya to work not only as a standalone application but also as a **compliance intelligence layer** for existing ERP systems, inspection applications, warehouses, manufacturers and e-commerce platforms.

14.1 Key Capabilities

The platform brings together:

- **Product and label image scanning**
- **OCR-based text extraction**
- **AI-based declaration extraction**
- **Package classification**
- **Applicability determination**
- **Version-aware Legal Metrology rule evaluation**
- **Potential violation detection**
- **Evidence and audit-trail management**
- **Human review workflow**
- **Structured compliance API responses**
- **Secure external-system integration**

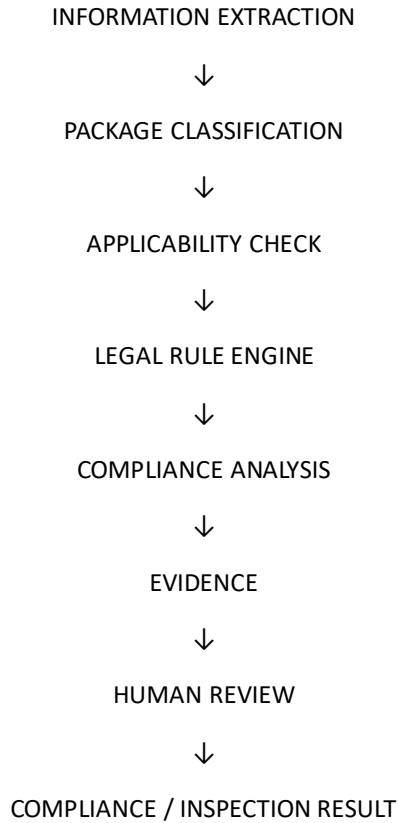
14.2 End-to-End Vision

PRODUCT IMAGE



OCR + AI



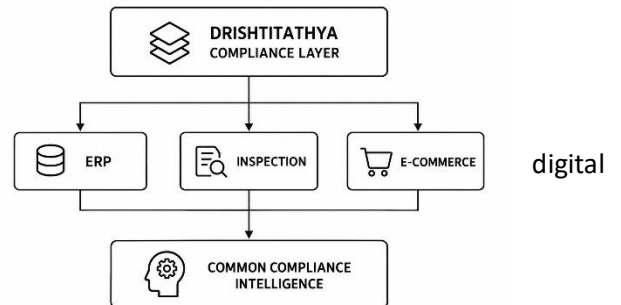


The central strength of the proposed system is that it does not stop at “**reading the label.**” It attempts to connect what is visible on the package with the **applicable legal requirements, evidence and review workflow.**

14.3 Integration Vision

Through the Scanner API:

This architecture allows organizations to adopt compliance capabilities without replacing their existing systems.



14.4 Responsible AI Approach

Because Legal Metrology compliance can involve legal interpretation and enforcement, DrishtiTathya should operate as an **AI-assisted decision-support system.**

The system should:

- Show the evidence supporting a finding.
- Identify the applicable rule.
- Provide confidence information.
- Distinguish potential violations from confirmed decisions.
- Route uncertain cases for human review.
- Maintain an audit trail of important actions.

14.5 Final Perspective

DrishtiTathya aims to transform packaged-commodity inspection from a largely manual process into a **digital, evidence-based and scalable workflow**.

Its long-term vision is:

“दृष्टि देखे, तथ्य परखे।”

“See the Label. Verify the Facts.”

The platform can serve as a bridge between **AI-based visual inspection, Legal Metrology requirements and practical compliance workflows**, helping consumers identify potential issues, helping businesses perform preventive self-audits, and helping authorities conduct more efficient and evidence-oriented inspections.

Conclusion: DrishtiTathya is envisioned not merely as a product-scanning application, but as a **compliance intelligence platform that connects vision, facts, legal rules and human verification into one integrated system**.